C7.2 Other cache maintenance considerations

This section describes the additional requirements that ensure correct local, domain, and downstream cache maintenance.

C7.2.1 Broadcast cache maintenance requirements

The master issuing a broadcast cache maintenance operation has to co-ordinate the following:

- appropriate action for local cache maintenance
- appropriate action for peer and downstream cache maintenance.

Issuing a broadcast cache maintenance transaction performs the required action on peer caches and causes the interconnect to generate a downstream cache maintenance transaction to other levels of cache.

The downstream cache maintenance transaction must be correctly ordered with respect to other transactions to the same cache line. The master carrying out the cache maintenance, must follow the sequence:

1. The master must complete any outstanding shareable transactions to a cache line before it issues a cache maintenance transaction to the same cache line.
   a. For CleanShared and CleanInvalid operations:
      If the master holds the cache line in a Dirty state it must issue a WriteBack or WriteClean transaction, that must complete, before issuing the cache maintenance transaction.

      Note

      If the cache line is initially clean, but there are outstanding cacheable transactions to the line, then it must be ensured that the line is not Dirty after the completion of all outstanding transactions to the cache line.

   b. For CleanInvalid and MakeInvalid transactions:
      After all outstanding transactions and required WriteBack or WriteClean transactions are complete, and before issuing the cache maintenance transaction, the master must invalidate the cache line.
2. After all outstanding transactions and required WriteBack or WriteClean transactions are complete, the master issues the appropriate cache maintenance transaction.

The master must not issue any further shareable transactions to the same cache line until the broadcast cache maintenance sequence is complete.

Note

All masters that support an external snoop filter must ensure that the information provided enables the snoop filter to correctly track the allocation of cache lines. Typically, this is ensured by the correct use of WriteBack and WriteClean transactions and the appropriate snoop responses. See *External snoop filter requirements* on page C10-273.

For a given memory location, cache maintenance operations are permitted to use different shareability and cacheability attributes to those that the page table attributes assign for any non-cache maintenance transaction to that location. This possible mismatch of attributes means that an interconnect cannot correctly determine the cacheability attributes to use for any interconnect-generated transactions that result from the cache maintenance operation, that are required if a snooped cache provides dirty data on the CD channel in response to a snoop transaction for the cache maintenance operation.

This specification recommends that, when an interconnect component has to generate a write to main memory as a result of receiving a dirty line during a cache maintenance operation, the interconnect component uses the Write-through No-allocate memory attribute for the transaction.

Copyright © 2003, 2004, 2010, 2011 ARM. All rights reserved.
Non-Confidential

Note

The use of the Write-through No-allocate attribute ensures that, if the line is allocated in a downstream cache, then that cache will be checked. This avoids stale data being held in that cache. If the line is not allocated in a downstream cache then using this memory attribute prevents the line from being allocated.

C7.2.2 Requirements for a snooped master

There are no additional requirements for a snooped master during a cache maintenance operation. All requirements are as specified in Chapter C5 *Snoop Transactions*.

C7.2.3 Processor cache maintenance instructions

The protocol requires that the cache maintenance operations use the **AxCACHE** and **AxDOMAIN** signals to identify the caches on which the cache maintenance operations must operate.

For a processor that has cache maintenance instructions that are required to operate on more or fewer caches than are defined by the **AxCACHE** and **AxDOMAIN** values, the cacheability and shareability of the transaction must be adapted to meet the requirements of the processor. For example, if a processor instruction performing a cache maintenance operation on a location with Device memory attributes is required to operate on all caches within the system then, the master must issue a cache maintenance transaction as Outer Shareable, since this is the most pervasive of the cache maintenance operations and operates on all the required caches.

C7.2.4 Unpredictable behavior with software cache maintenance

Cache maintenance can be used to reliably communicate shared memory data structures between a coherent group of masters and non-coherent agents. This process must follow a particular sequence to reliably make the data structures visible as required.

When using cache maintenance to make the writes of a non-coherent agent visible to a coherent group of masters there are periods of time when writing and reading the data structures gives UNPREDICTABLE results and can cause a loss of coherency.

The observation of a line that is being updated by a non-coherent agent is UNPREDICTABLE during the period between the clean transaction that starts the sequence and the invalidate transaction that completes it. During this period, it is permissible to see multiple transitions of a cache line that is being updated by a non-coherent agent.

Figure C7-1 shows the required sequence of communication between a coherent domain and a non-coherent agent.

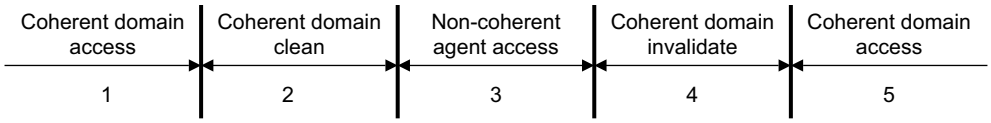


Figure C7-1 Required sequence of communication between coherent and non-coherent domains

The five stage sequence that Figure C7-1 shows is:

1. The coherent domain has access. During this stage, the coherent domain has full read and write access to the appropriate memory locations. This stage finishes when all required writes from the coherent domain are complete within the coherent domain.

2. The coherent domain is cleaned. During this stage, a cache clean operation is required for all the address locations that are undergoing software cache maintenance. The coherent domain clean forces all previous writes to be visible to the non-coherent agent. This stage finishes when all required writes are complete and therefore visible to the non-coherent agent.

3. The non-coherent agent has access. During this stage the non-coherent agent has both read and write access to the defined memory locations. This stage finishes when all required writes from the non-coherent agent are complete.

Copyright © 2003, 2004, 2010, 2011 ARM. All rights reserved.
Non-Confidential

4. The coherent domain is invalidated. During this stage, a cache invalidate operation is required for all the address locations that are undergoing software cache maintenance. This coherent domain invalidate stage removes all cached copies of the defined locations ensuring that any subsequent access from the coherent domain observes the writes from the non-coherent agent. This stage finishes when all the required invalidations are complete.

5. The coherent domain has full access to the defined memory locations.

Table C7-1 shows when accesses from the coherent domain or the non-coherent agent are permitted. The remaining accesses can have unpredictable results, with possible loss of coherency.

Table C7-1 Permitted accesses from the Coherent domain and non-coherent agent

Phase	Description	Coherent domain		External agent	
		Read	Write	Read	Write
1	Coherent domain access	Permitted	Permitted	-	-
2	Coherent domain clean	-	-	-	-
3	External agent access	-	-	Permitted	Permitted
4	Coherent domain invalidate	-	-	-	-
5	Coherent domain access	Permitted	Permitted	-	-

C7.2.5 Mismatched shareability and cacheability

To prevent a loss of coherency, the protocol requires that all accesses to a particular memory location must use the same values of **AxCACHE** and **AxDOMAIN**. This ensures that multiple agents accessing a particular memory location agree exactly on the caches in which the memory location might be held.

If agents accessing the same or overlapping locations do not agree exactly on the caches in which the location might be held, then this is defined as a software protocol error. See *Protocol errors* on page C1-150.

Copyright © 2003, 2004, 2010, 2011 ARM. All rights reserved.
Non-Confidential

Copyright © 2003, 2004, 2010, 2011 ARM. All rights reserved.
Non-Confidential

Chapter C8
Barrier Transactions

This chapter describes ACE barrier transactions. It contains the following sections:

- *About barrier transactions* on page C8-248
- *Barrier transaction signaling* on page C8-249
- *Barrier responses and domain boundaries* on page C8-251
- *Barrier requirements* on page C8-254.

Copyright © 2003, 2004, 2010, 2011 ARM. All rights reserved.
Non-Confidential